

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC – KỸ THUẬT MÁY TÍNH



Vi xử lý - Vi điều khiển

Báo cáo bài thực hành 1

Hiện thực hiệu ứng LED

GVHD : Phan Văn Sỹ

SV thực hiện : Trần Minh Quang – 2412855

TP. HỒ CHÍ MINH, THÁNG 9/ 2026



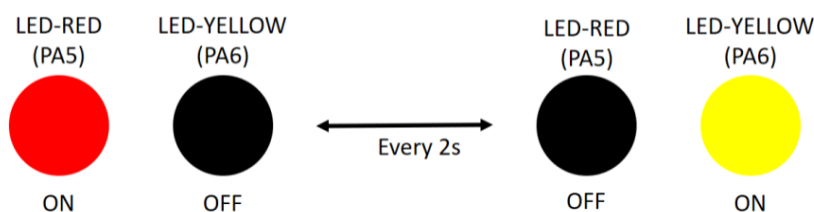
Mục lục

1 Bài tập 1	1
2 Bài tập 2	2
3 Bài tập 3	4
4 Bài tập 4	5
5 Bài tập 5	7
6 Bài tập 6	8
7 Bài tập 7	10
8 Bài tập 8	10
9 Bài tập 9	10
10 Bài tập 10	11
11 Mã nguồn cho bài thực hành 1	11

1 Bài tập 1

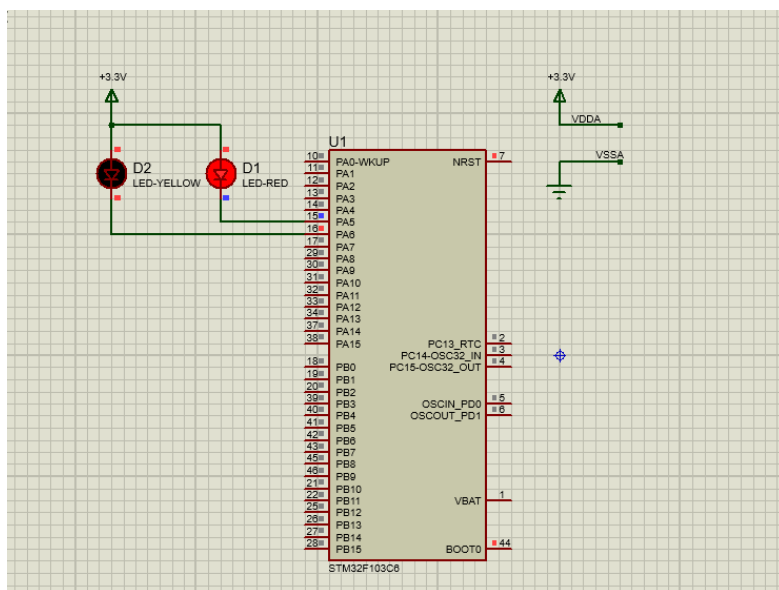
Từ sơ đồ mô phỏng trên Proteus, kết nối thêm một đèn LED vào chân PA6 của vi điều khiển STM32 (chân âm của LED được nối với chân PA6). Linh kiện được khuyến nghị dùng trong bài tập này là LED-YELLOW, có thể tìm thấy trong danh sách linh kiện của Proteus.

Trong bài tập này, trạng thái của hai đèn LED sẽ được hoán đổi cho nhau sau mỗi 2 giây, như minh họa trong hình dưới đây.



Hình 1: Chuyển đổi trạng thái cho 2 LED

Sơ đồ nguyên lý:



Hình 2: Sơ đồ nguyên lý cho bài tập 1

Link github cho file mô phỏng bài tập: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex1_blinkyled.pdsprj

Mã nguồn trong vòng lặp vô tận while:

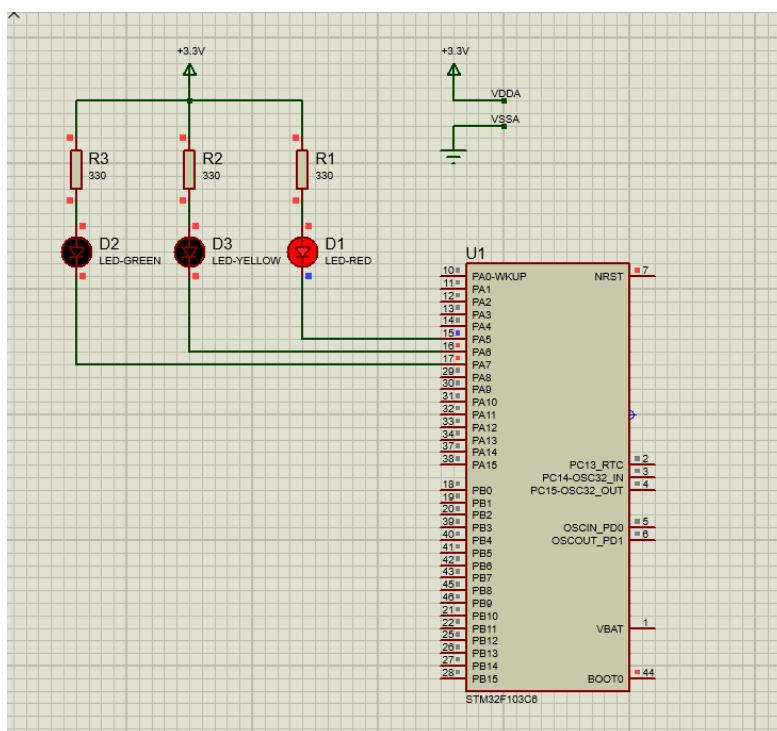
```
1  /* USER CODE BEGIN 2 */
2  int led_state = 0; // 0: red 1:yellow
3  int timer = 2; // chu kỳ 2s
4
5  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
6  HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
7  /* USER CODE END 2 */
8
9  /* Infinite loop */
10 /* USER CODE BEGIN WHILE */
11
12 while (1)
13 {
14     // active-low
15     HAL_Delay(1000);
16     timer--;
17     if(timer <= 0){
18         timer = 2; // reset cho chu kỳ mới
19         if(led_state == 0){
20             // red off yellow on
21             HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
22             HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);
23             led_state = 1;
24         }else{
25             // red on yellow off
26             HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
27             HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
28             led_state = 0;
29         }
30     }
31 }
32 /* USER CODE END WHILE */
```

Mã nguồn cho vòng while vô tận

2 Bài tập 2

Mở rộng bài tập 1 để mô phỏng hoạt động của đèn giao thông. Bổ sung một đèn LED thứ ba mang tên LED-GREEN vào hệ thống và kết nối với chân PA7. Chu kỳ sáng của hệ thống đèn giao thông này gồm: 5 giây cho đèn ĐỎ (RED), 2 giây cho đèn VÀNG (YELLOW) và 3 giây cho đèn XANH (GREEN). Đèn LED-GREEN cũng được điều khiển thông qua chân âm (active-low).

Sơ đồ nguyên lý:



Hình 3: Sơ đồ nguyên lý cho bài tập 2

Link github file mô phỏng cho bài tập 2: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex2_one-traffic-light.pdsprj

Khai báo kiểu dữ liệu typedef và mã nguồn chính trong vòng lặp vô tận while:

```
1  /* Private typedef -----*/
2  /* USER CODE BEGIN PTD */
3  typedef enum {
4      RED = 0,
5      GREEN,
6      YELLOW
7  } LED_STATE;
8  /* USER CODE END PTD */
9
10 /* USER CODE BEGIN 2 */
11 LED_STATE state = RED; // trạng thái ban đầu: đèn đỏ
12 int timer = 5;         // đèn đỏ sáng trong 5 giây
13
14 // active-low: RESET = LED on, SET = LED off
15 HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
16 HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET);
17 HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
18 /* USER CODE END 2 */
19
20 /* Infinite loop */
21 /* USER CODE BEGIN WHILE */
22 while (1)
23 {
24     timer--; // giảm bộ đếm mỗi giây
```

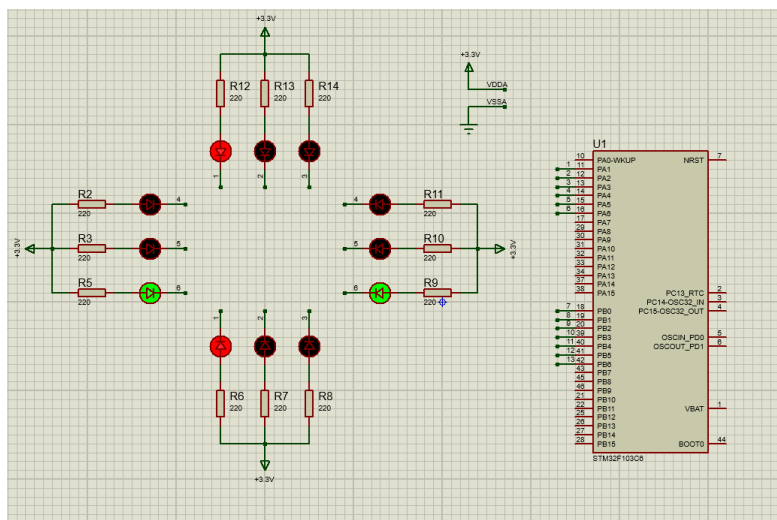
```
25
26 switch (state)
27 {
28     case RED:
29         // red on, green off, yellow off
30         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_RESET);
31         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET);
32         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
33
34         // sau 5s: RED -> GREEN
35         if (timer <= 0)
36         {
37             timer = 3;    // gr 3s
38             state = GREEN;
39         }
40         break;
41
42     case GREEN:
43         // red off, green on, yellow off
44         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
45         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_RESET);
46         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_SET);
47
48         // sau 3s: GREEN -> YELLOW
49         if (timer <= 0)
50         {
51             timer = 2;    // y 2s
52             state = YELLOW;
53         }
54         break;
55
56     case YELLOW:
57         // red off, green off, yellow on
58         HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, GPIO_PIN_SET);
59         HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin, GPIO_PIN_SET);
60         HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin, GPIO_PIN_RESET);
61
62         // sau 2s: YELLOW -> RED
63         if (timer <= 0)
64         {
65             timer = 5; // r 5s
66             state = RED;
67         }
68         break;
69     }
70
71     HAL_Delay(1000); // mỗi vòng lặp tương ứng 1 giây
72 }
73 /* USER CODE END WHILE */
```

Mã nguồn chương trình

3 Bài tập 3

Mở rộng hệ thống thành đèn giao thông ngã tư (4 hướng). Sắp xếp 12 đèn LED thành hình dạng hợp lý để mô phỏng hoạt động của hệ thống đèn giao thông ngã tư.

Sơ đồ nguyên lý:



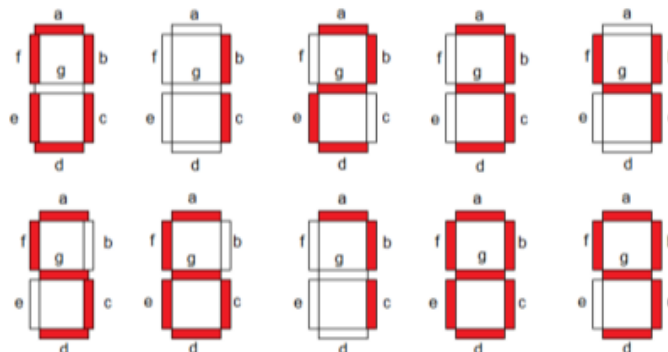
Hình 4: Sơ đồ nguyên lý cho bài tập 3

Link github file mô phỏng bài tập 3: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex3_4-way-traffic-light.pdsprj

4 Bài tập 4

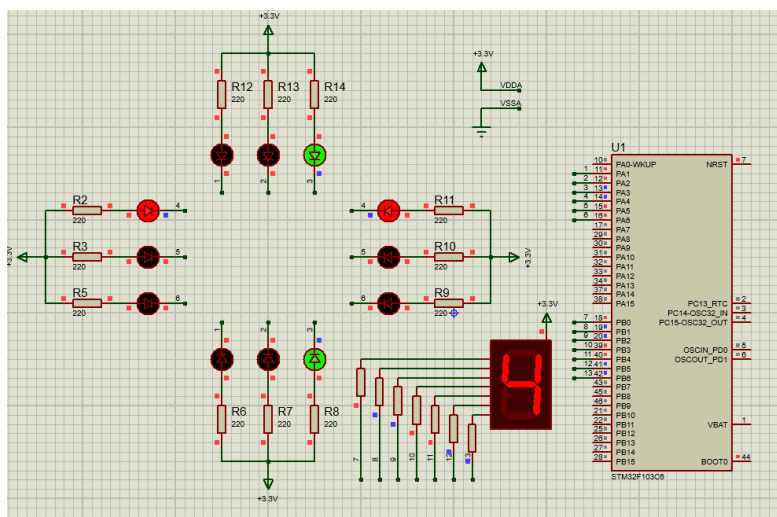
Bổ sung một LED 7 đoạn duy nhất vào sơ đồ nguyên lý của Bài tập 3. Linh kiện này có thể tìm thấy trong Proteus bằng từ khóa **7SEG-COM-ANODE**. Đối với linh kiện này, chân chung (common) cần được nối với nguồn cấp điện và các chân còn lại được kết nối lần lượt từ PB0 đến PB6. Do đó, để kích hoạt sáng một đoạn trên LED 7 đoạn, chân STM32 tương ứng cần ở mức logic 0 (0V).

Hiện thực một hàm có tên **display7SEG(int num)**. Giá trị đầu vào của hàm nằm trong khoảng từ 0 đến 9 và kết quả hiển thị tương ứng như minh họa dưới đây:



Hình 5: Sơ đồ chuyển trạng thái hiển thị của LED 7 đoạn

Sơ đồ nguyên lý:



Hình 6: Sơ đồ nguyên lý cho bài tập 4

Link github file mô phỏng bài tập 4: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex4_7-led-segment.pdsprj

Cài đặt cho hàm **display7SEG**, khởi tạo biến và mã nguồn chính trong vòng lặp while:

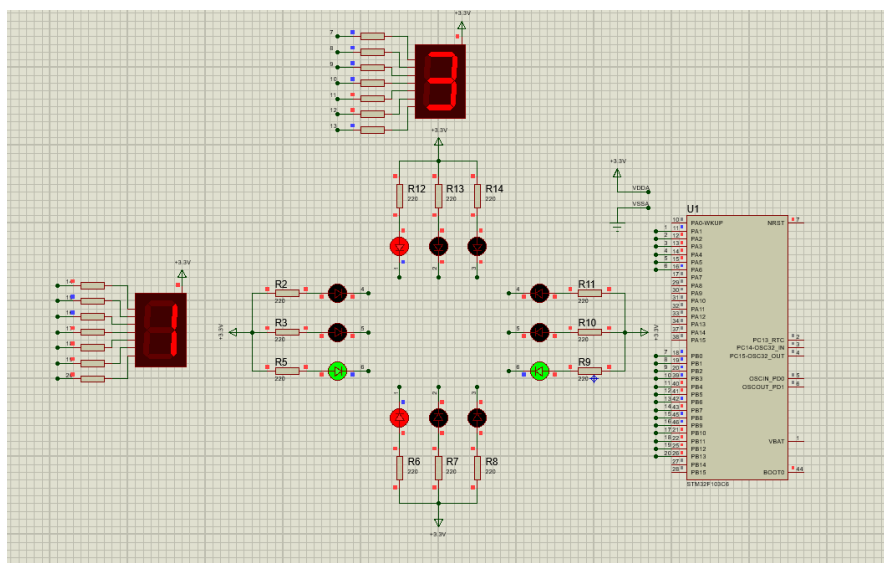
```
1  /* USER CODE BEGIN 0 */
2  void display7SEG(int num) {
3      // Mang ma hoa cho LED 7 đoạn Anode chung (g f e d c b a)
4      // 0: BAT (RESET), 1: TAT (SET)
5      const uint8_t segment_patterns[10] = {
6          0x3F, // 0: 0111111 -> sang a,b,c,d,e,f
7          0x06, // 1: 0000110 -> sang b,c
8          0x5B, // 2: 1011011 -> sang a,b,d,e,g
9          0x4F, // 3: 1001111 -> sang a,b,c,d,g
```

```
10     0x66, // 4: 1100110 -> sang b,c,f,g
11     0x6D, // 5: 1101101 -> sang a,c,d,f,g
12     0x7D, // 6: 1111101 -> sang a,c,d,e,f,g
13     0x07, // 7: 0000111 -> sang a,b,c
14     0x7F, // 8: 1111111 -> sang a,b,c,d,e,f,g
15     0x6F // 9: 1101111 -> sang a,b,c,d,f,g
16 };
17
18 if (num < 0 || num > 9) return;
19
20 uint8_t pattern = segment_patterns[num];
21
22 // Ghi trạng thái ra từng chân từ PB0 (a) đến PB6 (g)
23 // Anode chung: Bit = 1 thì xuất mức 0 (RESET) để đèn sáng
24 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, (pattern & (1 << 0)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
25 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, (pattern & (1 << 1)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
26 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_2, (pattern & (1 << 2)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
27 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, (pattern & (1 << 3)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
28 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, (pattern & (1 << 4)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
29 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, (pattern & (1 << 5)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
30 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, (pattern & (1 << 6)) ? GPIO_PIN_RESET : GPIO_PIN_SET);
31 }
32 /* USER CODE END 0 */
33
34 /* USER CODE BEGIN 2 */
35 int counter = 0;
36 /* USER CODE END 2 */
37
38 /* Infinite loop */
39 /* USER CODE BEGIN WHILE */
40 while (1)
41 {
42     if(counter >= 10) counter = 0;
43     display7SEG(counter++);
44     HAL_Delay(1000);
45 }
```

Mã nguồn chương trình

5 Bài tập 5

Tích hợp đèn LED 7 đoạn vào hệ thống đèn giao thông 4 ngã. Ở bài này, LED 7 đoạn được sử dụng để hiển thị giá trị đếm ngược thời gian của các đèn.

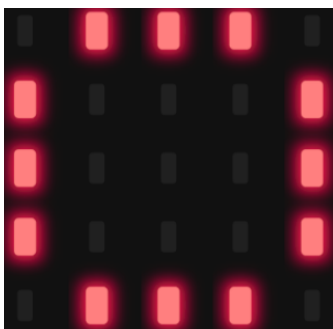


Hình 7: Sơ đồ nguyên lý cho bài tập 5

Link github file mô phỏng bài tập 5: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex5_count-down.pdsprj

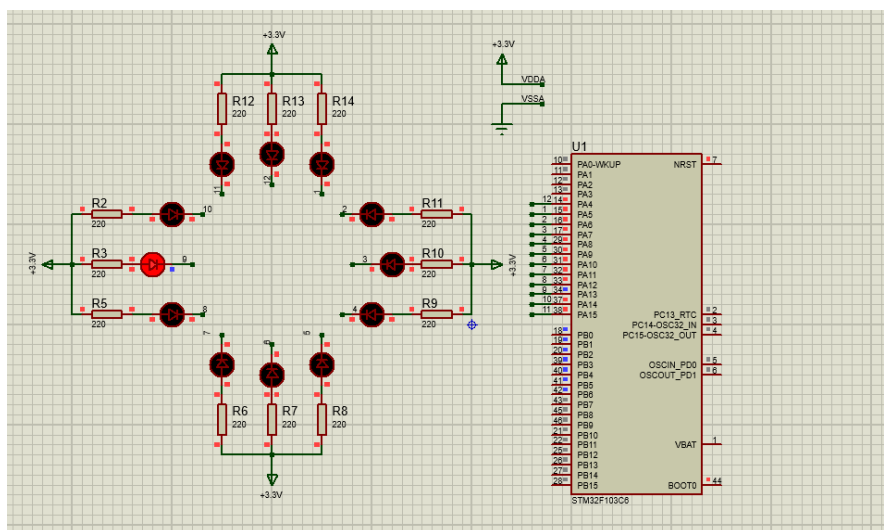
6 Bài tập 6

Trong bài tập này, một sơ đồ Proteus mới được thiết kế để mô phỏng đồng hồ kim (analog clock) với 12 số khác nhau. 12 đèn LED được kết nối lần lượt từ chân PA4 đến PA15 của STM32. Cách sắp xếp 12 đèn LED được trình bày như hình vẽ bên dưới.



Hình 8: Sơ đồ bố trí 12 LED cho đồng hồ kim

Sơ đồ nguyên lý:



Hình 9: Sơ đồ nguyên lý cho bài tập 6

Link github file mô phỏng cho bài tập 6: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex6_led-sequence.pdsprj

Chương trình kiểm thử tuần tự cho từng LED:

```
1  /* Private variables -----*/
2  /* USER CODE BEGIN PV */
3  // Khởi tạo chân đèn
4  const uint16_t CLOCK_PINS[12] = {
5      GPIO_PIN_4, // 0: Vị trí 12h
6      GPIO_PIN_5, // 1: Vị trí 1h
7      GPIO_PIN_6, // 2: Vị trí 2h
8      GPIO_PIN_7, // 3: Vị trí 3h
9      GPIO_PIN_8, // 4: Vị trí 4h
10     GPIO_PIN_9, // 5: Vị trí 5h
11     GPIO_PIN_10, // 6: Vị trí 6h
12     GPIO_PIN_11, // 7: Vị trí 7h
13     GPIO_PIN_12, // 8: Vị trí 8h
14     GPIO_PIN_13, // 9: Vị trí 9h
15     GPIO_PIN_14, // 10: Vị trí 10h
16     GPIO_PIN_15 // 11: Vị trí 11h
17 };
18 /* USER CODE END PV */
19
20 /* USER CODE BEGIN 2 */
21 // Tắt toàn bộ 12 LED trước khi bắt đầu (Active-low: SET là TAT)
22 for (int i = 0; i < 12; i++) {
23     HAL_GPIO_WritePin(GPIOA, CLOCK_PINS[i], GPIO_PIN_SET);
24 }
25 int current_led = 0;
26 /* USER CODE END 2 */
27
28 /* Infinite loop */
29 /* USER CODE BEGIN WHILE */
30 while (1)
31 {
32     // Tắt bóng LED ở bước trước
```

```
33     int prev_led = (current_led == 0) ? 11 : (current_led - 1);
34     HAL_GPIO_WritePin(GPIOA, CLOCK_PINS[prev_led], GPIO_PIN_SET);
35
36     // Bat bong LED hien tai (Active-low: RESET la BAT)
37     HAL_GPIO_WritePin(GPIOA, CLOCK_PINS[current_led], GPIO_PIN_RESET);
38
39     // 3Tang chi so bong den de chay vong tron 0-11
40     current_led = (current_led + 1) % 12;
41
42     HAL_Delay(1000); // 1s giay chuyen 1 bong
43 }
44 /* USER CODE END WHILE */
45
46 /* USER CODE BEGIN 3 */
47 /* USER CODE END 3 */
```

Hiện thực tuần tự 12 LED

7 Bài tập 7

Cài đặt cho hàm `clearAllClock()`. Dưới đây là mã nguồn:

```
1 void clearAllClock(void) {
2     HAL_GPIO_WritePin(GPIOA,
3         GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7 |
4         GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 |
5         GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15,
6         GPIO_PIN_SET); // SET de tat tat ca cac LED
7 }
```

Hàm `clearAllClock()`

8 Bài tập 8

Cài đặt cho hàm `setNumberOnClock(int num)`. Dưới đây là mã nguồn:

```
1 void setNumberOnClock(int num) {
2     if (num >= 0 && num < 12) {
3         HAL_GPIO_WritePin(GPIOA, CLOCK_PINS[num], GPIO_PIN_RESET);
4     }
5 }
```

Hàm `setNumberOnClock(int num)`

9 Bài tập 9

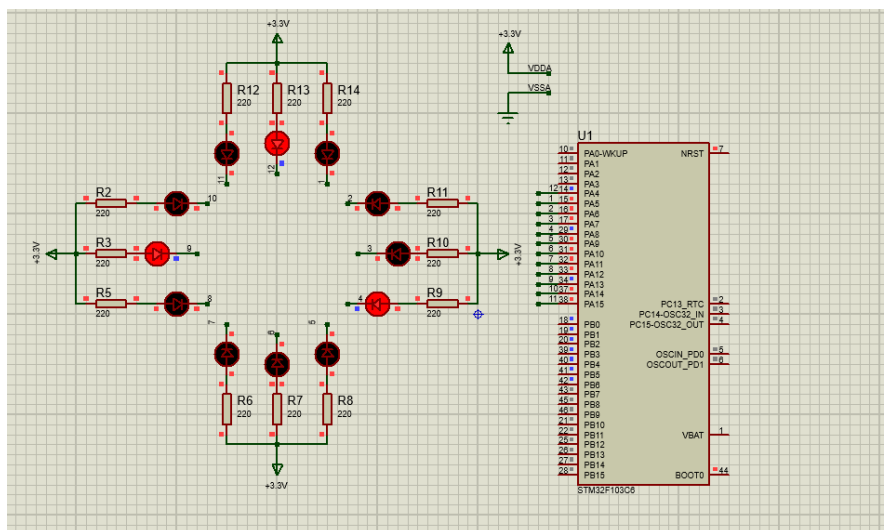
Cài đặt cho hàm `clearNumberOnClock(int num)`. Dưới đây là mã nguồn:

```
1 void clearNumberOnClock(int num) {  
2     if (num >= 0 && num < 12) {  
3         HAL_GPIO_WritePin(GPIOA, CLOCK_PINS[num], GPIO_PIN_SET);  
4     }  
5 }
```

Hàm `clearNumberOnClock(int num)`

10 Bài tập 10

Hệ thống đồng hồ sử dụng 12 LED. Tại thời điểm được khởi tạo để bắt đầu hiển thị thời gian, đồng hồ đang chỉ 09:24:00.



Hình 10: Chức năng hiển thị giờ/ phút/ giây cho đồng hồ 12 LED

Link github file mô phỏng cho bài tập 10: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab1/proteus_files/Ex10_12-led-clock.pdsprj

11 Mã nguồn cho bài thực hành 1

Toàn bộ mã nguồn cho các bài tập ở bài thực hành 1, với bài tập 7-8-9 được hiện thực chung ở thư mục bài tập 10: https://github.com/qanetta/Microprocessor-Lab/tree/main/lab1/stm32cube_src